

LAPORAN PRATIKUM

STRUKTUR DATA

Single Linked List

disusun Oleh:

Muhammad Yasin Habiburrahman

2511532016

Dosen Pengampu:

Dr. Wahyudi. S.T.M.T

Asisten Pratikum:

Sherly Sukmadira Putri



DEPARTEMEN INFORMATIKA FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia-Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman.

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pembimbing yang telah memberikan arahan, serta kepada seluruh anggota kelompok yang telah bekerja sama dengan baik sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami khususnya, dan bagi pembaca pada umumnya.

Padang, 5 Mei 2026

Muhammad Yasin Habiburrahman

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN.....	2
2.1 ADT Single Linked List.....	2
2.2 Menambahkan Data ke Single Linked List.....	2
2.2.1 Menambahkan Data ke Depan Linked List.....	2
2.2.2 Menambahkan Data ke Belakang Linked List	3
2.2.3 Mengambil Nilai Node.....	3
2.2.4 Menyisipkan Nilai pada Posisi Tertentu	3
2.2.5 Method Menampilkan Linked List	4
2.2.6 Method Main Sebagai Pengujian	5
2.3 Pencarian dalam Single Linked List	6
2.3.1 Fungsi untuk Memastikan Keberadaan Nilai	6
2.3.2 Method untuk Menampilkan Linked List	6
2.3.4 Method Main sebagai Pengujian.....	6
2.4 Penghapusan Elemen dalam Linked List	7
2.4.1 Menghapus Head dari Linked List.....	7
2.4.2 Menghapus Node Terakhir dari Linked List.....	7
2.4.3 Menghapus Node Berdasarkan Posisi.....	8
2.4.4 Method untuk Menampilkan Linked List	8
2.4.5 Method Main sebagai Pengujian.....	9
LAPORAN TUGAS PRAKTIKUM.....	10
ADT: Pasien.....	10
Driver : Rumah Sakit	11
BAB III KESIMPULAN.....	15
DAFTAR PUSTAKA.....	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Linked List adalah struktur data yang terdiri dari kumpulan node, di mana setiap node menyimpan dua hal — data itu sendiri dan pointer yang menunjuk ke node berikutnya. Berbeda dengan array yang elemen-elemennya tersimpan di lokasi memori yang berurutan, node-node pada Linked List dapat tersebar di lokasi memori mana saja karena keterhubungannya dijaga oleh pointer. Salah satu jenis Linked List yang paling dasar adalah Single Linked List (SLL), di mana setiap node hanya memiliki satu pointer yang menunjuk ke node berikutnya, dan node terakhir pointernya bernilai null sebagai tanda akhir dari list.

Dalam praktikum ini, dipelajari konsep Single Linked List beserta operasi-operasi dasarnya seperti penambahan node di depan, di belakang, dan pada posisi tertentu, penelusuran seluruh node (*traversal*), pencarian data (*searching*), serta penghapusan node dari berbagai posisi. Pemahaman terhadap SLL menjadi fondasi penting sebelum mempelajari variasi Linked List yang lebih kompleks seperti Double Linked List dan Circular Linked List.

1.2 Tujuan

1. Memahami konsep Single Linked List dan cara kerja pointer dalam menghubungkan antar node.
2. Mampu mengimplementasikan operasi penambahan node di depan, di belakang, dan pada posisi tertentu dalam SLL.
3. Mampu mengimplementasikan operasi penelusuran, pencarian, dan penghapusan node pada SLL.
4. Memahami perbedaan antara Linked List dan array dalam hal alokasi memori dan efisiensi operasi.

1.3 Manfaat Praktikum

1. Praktikan mampu menerapkan konsep Single Linked List dalam menyelesaikan permasalahan pemrograman yang membutuhkan struktur data dinamis dengan operasi penyisipan dan penghapusan yang efisien.
2. Praktikan memahami konsep pointer dan referensi objek di Java secara lebih mendalam melalui implementasi node yang saling terhubung.
3. Praktikan memiliki pemahaman yang kuat mengenai Linked List sebagai dasar untuk mempelajari struktur data lanjutan seperti Double Linked List, Circular Linked List, Tree, dan Graph.

BAB II

PEMBAHASAN

2.1 ADT Single Linked List

```
1 package pekan5_2511532016;
2
3 public class NodeSLL_2511532016 {
4     int data_2016;
5     NodeSLL_2511532016 next_2016;
6     public NodeSLL_2511532016(int data_2016) {
7         this.data_2016 = data_2016;
8         this.next_2016 = null;
9     }
10
11 }
```

Kode Program 2.1 – ADT SLL

NodeSLL_2511532016 adalah block paling dasar dari seluruh struktur Single Linked List. Class ini merepresentasikan satu buah node yang terdiri dari dua komponen, yaitu data_2016 bertipe int untuk menyimpan nilai, dan next_2016 bertipe NodeSLL_2511532016 itu sendiri; yang merupakan pointer ke node berikutnya. Di konstruktor, next_2016 diinisialisasi null karena node yang baru dibuat belum terhubung ke node manapun.

Visualisasi dari single linked list dapat dilihat sebagai berikut:

[data1 next] --> [data2 next] --> [data3 next] --> null

2.2 Menambahkan Data ke Single Linked List

2.2.1 Menambahkan Data ke Depan Linked List

```
5 public static NodeSLL_2511532016 insertAtFront_2016(NodeSLL_2511532016 head_2016, int value_2016) {
6     NodeSLL_2511532016 new_node_2016 = new NodeSLL_2511532016(value_2016);
7     new_node_2016.next_2016 = head_2016;
8     return new_node_2016;
9 }
```

Kode Program 2.2 – Fungsi insertAtFront

Fungsi ini mengembalikan node dengan tipe data NodeSLL_2511532016, dengan parameter head bertipe NodeSLL_2511532016 dan suatu bilangan yang ingin dimasukkan nilainya. Fungsi ini akan menambahkan nilai baru di depan linked list dan akan menjadi head barunya. Head yang sebelumnya menjadi pointer dari head yang baru.

2.2.2 Menambahkan Data ke Belakang Linked List

```

11 public static NodeSLL_2511532016 insertAtEnd_2016(NodeSLL_2511532016 head_2016, int value_2016) {
12     //Buat sebuah node dengan sebuah nilai
13     NodeSLL_2511532016 newNode_2016 = new NodeSLL_2511532016(value_2016);
14     //Jika list kosong maka node jadi head
15     if(head_2016 == null){
16         return newNode_2016;
17     }
18     //Simpan head ke variabel sementara
19     NodeSLL_2511532016 last_2016 = head_2016;
20     //Telusuri ke node akhir
21     while(last_2016.next_2016 != null){
22         last_2016 = last_2016.next_2016;
23     }
24     //Ubah pointer
25     last_2016.next_2016 = newNode_2016;
26     return head_2016;
27 }

```

Kode Program 2.3 – Fungsi insertAtBack

Fungsi ini menerima dua parameter, yaitu head dengan tipe node, dan value yang bertipe int. Awalnya, variabel value akan diubah menjadi node. Kemudian, dilakukan pengecekan terhadap linked list. Jika tidak ada elemen (kosong), variabel value akan menjadi head nya.

Jika tidak kosong, suatu variabel dengan tipe data node, last_2016, akan menyimpan nilai head sebagai titik awal. Dia akan menelusuri linked list sampai ujung linked list, yang ditandai dengan pointer ke node selanjutnya yang bernilai null. Jika sudah sampai null, ia akan berhenti mencari dan otomatis berhenti pada bagian paling belakang dari linked list. Barulah nilainya diubah sesuai dengan variabel value.

2.2.3 Mengambil Nilai Node

```

29 static NodeSLL_2511532016 GetNode_2016(int data_2016){
30     return new NodeSLL_2511532016(data_2016);
31 }

```

Kode Program 2.4 – Fungsi GetNode

Method sederhana yang hanya membuat dan mengembalikan node baru. Nantinya akan dipakai oleh insertPos agar pembuatan node tidak perlu ditulis ulang.

2.2.4 Menyisipkan Nilai pada Posisi Tertentu

```

33 static NodeSLL_2511532016 insertPos_2016(NodeSLL_2511532016 headNode_2016, int position_2016, int value_2016) {
34     NodeSLL_2511532016 head_2016 = headNode_2016;
35     if(position_2016 < 1) System.out.println("Invalid Position");
36     if(position_2016 == 1) {
37         NodeSLL_2511532016 new_node_2016 = new NodeSLL_2511532016(value_2016);
38         new_node_2016.next_2016 = head_2016;
39         return new_node_2016;
40     }
41     else {
42         while(position_2016-- != 0) {
43             if(position_2016 == 1) {
44                 NodeSLL_2511532016 newNode_2016 = GetNode_2016(value_2016);
45                 newNode_2016.next_2016 = headNode_2016.next_2016;
46                 headNode_2016.next_2016 = newNode_2016;
47                 break;
48             }
49             headNode_2016 = headNode_2016.next_2016;
50         }
51         if(position_2016 != 1) System.out.println("Posisi di luar jangkauan");
52         return head_2016;
53     }
54 }

```

Kode Program 2.5 – Fungsi insertPos

Fungsi ini digunakan untuk menambahkan nilai pada posisi yang diinginkan. Sebelumnya, node headNode yang diberikan akan disimpan pada variabel sementara, yaitu head.

Terdapat 3 kasus. Pertama ketika posisi yang diberikan bernilai <1 , ini akan dihandle dengan menampilkan “Invalid Position” ke layar. Kemudian ketika diberikan input posisi=1. Hal ini sama saja dengan fungsi insertAtFront().

Untuk posisi lainnya, nilai dari posisi yang diberikan akan di-*decrement* hingga nilainya 1. Selain di-*decrement*, headNode akan menelusuri node-node di sampingnya. Jika nilai posisi sudah menjadi 1, dibuat suatu node baru dengan nilai sesuai input dan headNode akan menjadikan node baru ini sebagai nilai node selanjutnya.

2.2.5 Method Menampilkan Linked List

```

57 public static void printList_2016(NodeSLL_2511532016 head) {
58     NodeSLL_2511532016 curr_2016 = head;
59     while(curr_2016.next_2016 != null) {
60         System.out.print(curr_2016.data_2016+"-->");
61         curr_2016 = curr_2016.next_2016;
62     }
63     if(curr_2016.next_2016 == null) {
64         System.out.print(curr_2016.data_2016);
65         System.out.println();
66     }
67 }

```

Kode Program 2.6 – Method printList

Method ini sangat sederhana, yaitu menampilkan nilai-nilai pada linked list. Ia akan mencetak semua nilai ,dengan tambahan tanda panah “→”, selagi nilai node di sebelahnya belum null. Jika null, nilai node terakhir akan dicetak secara terpisah karena nilainya tidak dapat dicetak pada perulangan while sebelumnya.

2.2.6 Method Main Sebagai Pengujian

```

70● public static void main(String[] args) {
71     //Buat linked list
72     NodeSLL_2511532016 head_2016 = new NodeSLL_2511532016(2);
73     head_2016.next_2016 = new NodeSLL_2511532016(3);
74     head_2016.next_2016.next_2016 = new NodeSLL_2511532016(5);
75     head_2016.next_2016.next_2016.next_2016 = new NodeSLL_2511532016(6);
76
77     System.out.print("Senarai berantai awal: ");
78     printList_2016(head_2016);
79
80     System.out.print("Tambah 1 simpul di depan: ");
81     int data_2016 = 1;
82     head_2016 = insertAtFront_2016(head_2016, data_2016);
83     printList_2016(head_2016);
84
85     System.out.print("Tambah 1 simpul di belakang: ");
86     int data2_2016 = 7;
87     head_2016 = insertAtEnd_2016(head_2016, data2_2016);
88     printList_2016(head_2016);
89
90     System.out.print("Tambah 1 simpul ke data 4: ");
91     int data3_2016 = 4;
92     int pos_2016 = 4;
93     head_2016 = insertPos_2016(head_2016, pos_2016, data3_2016);
94
95     printList_2016(head_2016);
96 }
97 }

```

Kode Program 2.7 – Method Main untuk Menguji Memasukkan Data ke Linked List

Untuk pengujian, dibuat suatu node baru dengan nilai head 2, dan nilai-nilai node disebelahny adalah 3.5 dan 6 berturut-turut. Kemudian nilai linked list akan ditampilkan. Saat ini adalah 2→3→5→6. Kemudian node baru dengan nilai 1 ditambahkan dengan menggunakan fungsi insertAtFront() untuk menggantikan nilai head. Linked list saat ini adalah 1→2→3→5→6.

Kemudian, digunakan fungsi insertArEnd() untuk menambahkan node baru dengan nilai 7. Linked list saat ini adalah 1→2→3→5→6→7.

Terakhir, fungsi inseertPos() akan digunakan untuk menyisipkan node dengan nilai 4 pada posisi ke-4. Hasil akhir linked list adalah 1→2→3→4→5→6→7.

Output pada terminal adalah sebagai berikut

```

Senarai berantai awal: 2-->3-->5-->6
Tambah 1 simpul di depan: 1-->2-->3-->5-->6
Tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
Tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7

```

Gambar 2.1 – Output Pemasukan Data

2.3 Pencarian dalam Single Linked List

2.3.1 Fungsi untuk Memastikan Keberadaan Nilai

```

4 static boolean searchKey_2016(NodeSLL_2511532016 head_2016, int key_2016) {
5     NodeSLL_2511532016 curr_2016 = head_2016;
6     while(curr_2016 != null) {
7         if(curr_2016.data_2016 == key_2016)
8             return true;
9         curr_2016 = curr_2016.next_2016;}
10    return false;
11    }

```

Kode Program 2.7 – Fungsi searchKey

Fungsi ini mencari nilai dari variabel key secara linear dalam linked list. Jika ditemukan, akan mengembalikan nilai true. Jika tidak, akan mengembalikan nilai false.

2.3.2 Method untuk Menampilkan Linked List

```

14 public static void traversal_2016(NodeSLL_2511532016 head_2016) {
15     NodeSLL_2511532016 curr_2016 = head_2016;
16     while(curr_2016 != null) {
17         System.out.print(" " + curr_2016.data_2016);
18         curr_2016 = curr_2016.next_2016;}
19     System.out.println();
20 }

```

Kode Program 2.8 – Method Travesal

Method ini hanya menampilkan isi dari linked list dengan cara menelusuri isinya dari head sampai menemukan nilai null dan mencetaknya satu per satu.

2.3.4 Method Main sebagai Pengujian

```

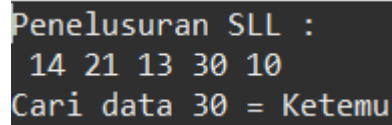
23 public static void main(String[] args) {
24     NodeSLL_2511532016 head_2016 = new NodeSLL_2511532016(14);
25     head_2016.next_2016 = new NodeSLL_2511532016(21);
26     head_2016.next_2016.next_2016 = new NodeSLL_2511532016(13);
27     head_2016.next_2016.next_2016.next_2016 = new NodeSLL_2511532016(30);
28     head_2016.next_2016.next_2016.next_2016.next_2016 = new NodeSLL_2511532016(10);
29
30     System.out.println("Penelusuran SLL : ");
31     traversal_2016(head_2016);
32
33     int key = 30;
34     System.out.print("Cari data " + key + " = ");
35     if(searchKey_2016(head_2016, key)) System.out.print("Ketemu");
36     else System.out.print("Tidak ada");
37
38 }

```

Kode Program 2.9 – Method Main untuk Pengujian Pencarian Data dalam Linked List

Pertama, list dibuat dengan 5 node yang bernilai 14, 21, 13, 30, dan 10 secara berurutan. Kemudian, method traversal() dipanggil untuk menampilkan isi linked

list. Barulah fungsi `searchKey()` mencari nilai dari key yang diminta, yaitu 30. Keluaran dari program tersebut adalah sebagai berikut



```

Penelusuran SLL :
14 21 13 30 10
Cari data 30 = Ketemu
  
```

Gambar 2.2 – Output Pencarian Data

2.4 Penghapusan Elemen dalam Linked List

2.4.1 Menghapus Head dari Linked List

```

6 public static NodeSLL_2511532016 deleteHead_2016(NodeSLL_2511532016 head_2016) {
7     if(head_2016 == null) {
8         return null;
9     }
10    head_2016 = head_2016.next_2016;
11    return head_2016;
12 }
  
```

Kode Program 2.10 – Fungsi deleteHead

Fungsi ini menghapus head dari linked list jika tidak bernilai null, dan posisi head akan diberikan kepada node selanjutnya dalam list

2.4.2 Menghapus Node Terakhir dari Linked List

```

16 public static NodeSLL_2511532016 removeLastNode_2016(NodeSLL_2511532016 head_2016) {
17     if(head_2016 == null) return null;
18     if(head_2016.next_2016 == null) return null;
19
20     NodeSLL_2511532016 secondLast_2016 = head_2016;
21     while(secondLast_2016.next_2016.next_2016 != null) {
22         secondLast_2016 = secondLast_2016.next_2016;
23     }
24     secondLast_2016.next_2016 = null;
25     return null;
26 }
  
```

Kode Program 2.11 – Fungsi removeLastNode

Sebelum mencoba untuk menghapus node terakhir, dilakukan pengecekan apakah head node atau node di sebelahnya bernilai null. Jika iya, kembalikan nilai null.

Jika tidak, dibuat sebuah node baru dengan nama `secondLast` dan head node sebagai pointer awalnya. Kemudian, dilakukan traversal dari head node menuju node kedua dari belakang yang ditandai dengan dua node setelah node tersebut bernilai null. Jika sudah ditemukan, nilai node berikutnya otomatis adalah node terakhir. Nilainya akan diubah menjadi null oleh `secondLast`.

2.4.3 Menghapus Node Berdasarkan Posisi

```

29● public static NodeSLL_2511532016 deleteNode(NodeSLL_2511532016 head_2016, int position_2016) {
30     NodeSLL_2511532016 temp_2016 = head_2016;
31     NodeSLL_2511532016 prev_2016 = null;
32
33     //Null linked list
34     if(temp_2016 == null) return head_2016;
35
36     //Head dihapus
37●   if(position_2016 == 1) {
38         head_2016 = temp_2016.next_2016;
39         return head_2016;
40     }
41
42●   for(int i_2016 = 1; temp_2016 != null && i_2016 < position_2016; i_2016++) {
43       prev_2016 = temp_2016;
44       temp_2016 = temp_2016.next_2016;
45
46       if(temp_2016 != null) prev_2016.next_2016 = temp_2016.next_2016;
47       else System.out.println("Data tidak valid");
48       return head_2016;
49   }
50 }

```

Kode Program 2.12 – Fungsi deleteNode

Pertama akan dibuat sepasang variabel baru, temp dan prev, dengan head node sebagai pointer untuk variabel temp, dan prev yang bernilai null..

Terdapat tiga kasus, yang pertama adalah jika linked list kosong, maka akan mengembalikan null. Kemudian ketika posisi yang ingin dihapus adalah posisi pertama, ini sama saja dengan fungsi deleteHead(). Untuk posisi di tengah, temp dan prev menelusuri linked list. Saat temp sampai di posisi yang dituju, pointer prev.next diarahkan langsung ke temp.next sehingga node tersebut terlewat dan terputus dari list.

2.4.4 Method untuk Menampilkan Linked List

```

53● public static void printList_2016(NodeSLL_2511532016 head) {
54     NodeSLL_2511532016 curr_2016 = head;
55●   while(curr_2016.next_2016 != null) {
56         System.out.print(curr_2016.data_2016+"-->");
57         curr_2016 = curr_2016.next_2016;
58     }
59●   if(curr_2016.next_2016 == null) {
60         System.out.print(curr_2016.data_2016);
61         System.out.println();
62     }
63 }

```

Kode Program 2.13 – Method printtList

Fungsi ini sama persis dengan fungsi yang ada pada kelas TambahSLL, yaitu menampilkan isi dari linked list dengan diikuti tanda → untuk setiap node nya.

2.4.5 Method Main sebagai Pengujian

```

66● public static void main(String[] args) {
67     NodeSLL_2511532016 head_2016 = new NodeSLL_2511532016(1);
68     head_2016.next_2016 = new NodeSLL_2511532016(2);
69     head_2016.next_2016.next_2016 = new NodeSLL_2511532016(3);
70     head_2016.next_2016.next_2016.next_2016 = new NodeSLL_2511532016(4);
71     head_2016.next_2016.next_2016.next_2016.next_2016 = new NodeSLL_2511532016(5);
72     head_2016.next_2016.next_2016.next_2016.next_2016.next_2016 = new NodeSLL_2511532016(6);
73
74     System.out.println("List awal: ");
75     printList_2016(head_2016);
76
77     head_2016 = deleteHead_2016(head_2016);
78     System.out.println("List setelah head dihapus: ");
79     printList_2016(head_2016);
80
81     int position_2016 = 2;
82     head_2016 = deleteNode(head_2016, position_2016);
83     System.out.println("List setelah node 2 dihapus: ");
84     printList_2016(head_2016);
85 }

```

Kode Program 2.14 – Method Main untuk Pengujian Penghapusan Data dalam Linked List

Awalnya, dimasukkan angka 1 hingga 6 dalam list, sehingga membentuk list berikut $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$. Kemudian, bagian paling depan dihapus dengan menggunakan fungsi `deleteHead`, sehingga list menjadi $2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$. Lalu, fungsi `deleteNode` digunakan untuk menghapus data di posisi ke-2. Di posisi ke-2, datanya adalah 3, sehingga hasil akhir list adalah $2 \rightarrow 4 \rightarrow 5 \rightarrow 6$.

```

List awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah node 2 dihapus:
2-->4-->5-->6

```

Gambar 2.3 – Output Penghapusan Data

LAPORAN TUGAS PRAKTIKUM

ADT: Pasien

```
1 package pekan5_2511532016;
2 public class Pasien_2511532016 {
3     String namaPasien_2016;
4     String penyakit_2016;
5     int nomorAntrian_2016;
6
7     Pasien_2511532016 next_2016;
8     public Pasien_2511532016() {
9         this.namaPasien_2016 = null;
10        this.penakit_2016 = null;
11        this.next_2016 = null;
12    }
13    public String getNamaPasien() {
14        return namaPasien_2016;
15    }
16    public String getPenyakit() {
17        return penyakit_2016;
18    }
19    public int getNomorAntrian() {
20        return nomorAntrian_2016;
21    }
22    public Pasien_2511532016 getNext() {
23        return next_2016;
24    }
25
26    public void setNamaPasien(String namaPasien_2016) {
27        this.namaPasien_2016 = namaPasien_2016;
28    }
29    public void setPenyakit(String penyakit_2016) {
30        this.penakit_2016 = penyakit_2016;
31    }
32    public void setNomorAntrian(int nomorAntrian_2016) {
33        this.nomorAntrian_2016 = nomorAntrian_2016;
34    }
35    public void setNext(Pasien_2511532016 next_2016) {
36        this.next_2016 = next_2016;
37    }
38 }
39
40 }
```

ADT Pasien dengan Implementasi Single Linked List

ADT ini adalah sebuah node yang akan digunakan untuk mengimplementasikan single linked list nantinya. Atribut yang dimilikinya adalah namaPasien, penyakit, dan nomorAntrian. Atribut next yang dimiliki setiap pasien digunakan sebagai pointer untuk pasien berikutnya dalam antrian. Kemudian terdapat setter dan getter untuk setiap atribut yang dimiliki oleh pasien.

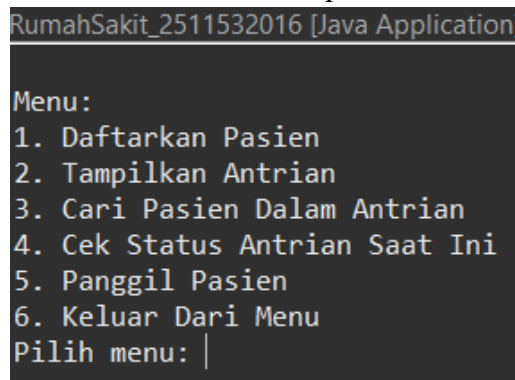
Driver : Rumah Sakit

```

5 public class RumahSakit_2511532016 {
6     static Scanner sc = new Scanner(System.in);
7     static int nomorAntrian=0;
8
9     static void tampilkanMenu_2016() {
10         System.out.println("\nMenu:");
11         System.out.println("1. Daftarkan Pasien");
12         System.out.println("2. Tampilkan Antrian");
13         System.out.println("3. Cari Pasien Dalam Antrian");
14         System.out.println("4. Cek Status Antrian Saat Ini");
15         System.out.println("5. Panggil Pasien");
16         System.out.println("6. Keluar Dari Menu");
17     }

```

Method untuk Menampilkan Menu



```

RumahSakit_2511532016 [Java Application]

Menu:
1. Daftarkan Pasien
2. Tampilkan Antrian
3. Cari Pasien Dalam Antrian
4. Cek Status Antrian Saat Ini
5. Panggil Pasien
6. Keluar Dari Menu
Pilih menu: |

```

Tampilan Menu

Awalnya, dideklarasikan dua buah variabel secara global, yaitu nomorAntrian untuk menyimpan nomorAntrian secara tetap, dan scanner untuk membaca input. Kemudian terdapat method tampilkanMenu untuk menampilkan menu.

```

19 static Pasien_2511532016 DaftarkanPasien_2016(Pasien_2511532016 head_2016) {
20     Pasien_2511532016 pasienBaru_2016 = new Pasien_2511532016();
21     System.out.print("Masukkan Nama Pasien: ");
22     String namaPasien = sc.nextLine();
23     pasienBaru_2016.setNamaPasien(namaPasien);
24     System.out.print("Masukkan Penyakit Pasien: ");
25     String penyakit_2016 = sc.nextLine();
26     pasienBaru_2016.setPenyakit(penyakit_2016);
27     pasienBaru_2016.setNomorAntrian(++nomorAntrian);
28     if (head_2016 == null) {
29         System.out.println("Pasien Berhasil Didaftarkan!");
30         return pasienBaru_2016;
31     }
32     Pasien_2511532016 temp_2016 = head_2016;
33     while (temp_2016.getNext() != null) {
34         temp_2016 = temp_2016.getNext();
35     }
36     temp_2016.setNext(pasienBaru_2016);
37     System.out.println("Pasien Berhasil Didaftarkan!");
38     return head_2016;
39 }

```

Fungsi untuk Mendaftarkan Pasien

Fungsi ini akan membuat sebuah objek pasien dengan nama pasienBaru. Kemudian nama dan penyakit pasien akan dimasukkan oleh pengguna. Nomor antrian akan diberikan secara otomatis untuk setiap pasien.

Jika antrian kosong, maka pasien tersebut akan menjadi pasien pertama dalam antrian. Jika tidak, akan dibuat variabel baru dengan head sebagai pointer awal, lalu ia akan bergerak dalam antrian hingga menemukan node null pada node berikutnya. Jika sudah ditemukan, maka pasien baru akan mengisi bagian null tersebut, dan akan menjadikan node berikutnya null untuk tempat pasien berikutnya.

```
Pilih menu: 1
Masukkan Nama Pasien: Suheng
Masukkan Penyakit Pasien: Kepala gatal
Pasien Berhasil Didaftarkan! Nomor antrian: 1

Menu:
1. Daftarkan Pasien
2. Tampilkan Antrian
3. Cari Pasien Dalam Antrian
4. Cek Status Antrian Saat Ini
5. Panggil Pasien
6. Keluar Dari Menu
Pilih menu: 1
Masukkan Nama Pasien: Sugeng
Masukkan Penyakit Pasien: Hidung gatal
Pasien Berhasil Didaftarkan! Nomor antrian: 2
```

Tampilan Opsi 1

```
41 static void TampilkanAntrian(Pasien_2511532016 head_2016) {
42     if(head_2016 == null) {
43         System.out.println("Antrian Kosong!");
44         return;
45     }
46     Pasien_2511532016 temp_2016 = head_2016;
47     while(temp_2016 != null) {
48         System.out.println("\nNama Pasien : " + temp_2016.getNamaPasien());
49         System.out.println("Penyakit Pasien : " + temp_2016.getPenyakit());
50         System.out.println("Nomor Antrian : " + temp_2016.getNomorAntrian());
51         System.out.println();
52         temp_2016 = temp_2016.getNext();
53     }
54 }
```

Method TampilkanAntrian

Method ini digunakan untuk menampilkan antrian saat ini. Pada tampilan antrian, akan berisikan daftar nama, penyakit, dan nomor antrian pasien yang sedang mengantri. Akan menampilkan “Antrian Kosong” ketika head bernilai null, atau dapat dikatakan tidak ada pasien yang sedang mengantri.

```
Pilih menu: 2
|
Nama Pasien      : Suheng
Penyakit Pasien  : Kepala gatal
Nomor Antrian    : 1

Nama Pasien      : Sugeng
Penyakit Pasien  : Hidung gatal
Nomor Antrian    : 2
```

Tampilan Opsi 2

```
56 static void CariPasien(Pasien_2511532016 head_2016) {
57     if (head_2016 == null) {
58         System.out.println("Antrian Kosong!");
59         return;
60     }
61     System.out.print("Masukkan nama pasien yang dicari: ");
62     String nama_2016 = sc.nextLine().toLowerCase();
63
64     Pasien_2511532016 temp_2016 = head_2016;
65     while (temp_2016 != null) {
66         if (temp_2016.getNamaPasien().toLowerCase().equals(nama_2016)) {
67             System.out.println("Pasien Ada!");
68             return;
69         }
70         temp_2016 = temp_2016.getNext();
71     }
72
73     System.out.println("Pasien Tidak Ada!");
74 }
```

Method untuk Mencari Pasien

Method ini digunakan untuk melihat apakah pasien berada dalam antrian atau tidak. Jika antrian kosong, yang ditandai dengan `head==null`, maka akan menampilkan “Antrian Kosong”. Jika tidak, maka akan dicari nama pasien sesuai dengan input yang dimasukkan pengguna. Nama pasien akan dikonversi menjadi String non-kapital untuk memudahkan pencarian.

Jika ditemukan, akan menampilkan “Pasien Ada!”. Jika tidak, akan menampilkan “Pasien Tidak Ada!”.

```
Pilih menu: 3
Masukkan nama pasien yang dicari: sUhEnG
Pasien Ada!

Menu:
1. Daftarkan Pasien
2. Tampilkan Antrian
3. Cari Pasien Dalam Antrian
4. Cek Status Antrian Saat Ini
5. Panggil Pasien
6. Keluar Dari Menu
Pilih menu: 3
Masukkan nama pasien yang dicari: sUheNdaR
Pasien Tidak Ada!
```

Tampilan Opsi 3


```

76 static void CekStatusAntrian(Pasien_2511532016 head_2016) {
77     if(head_2016== null) {
78         System.out.println("Antrian Kosong!");
79         return;
80     }
81     Pasien_2511532016 temp_2016 = head_2016;
82     int i_2016 =0;
83     while(temp_2016 != null) {
84         i_2016++;
85         temp_2016 = temp_2016.getNext();
86     }
87     System.out.println("\nSaat ini, terdapat " + i_2016 + " pasien.");
88
89     System.out.println("Status Pasien Selanjutnya: ");
90     System.out.println("Nama Pasien      : " + head_2016.getNamaPasien());
91     System.out.println("Penyakit Pasien : " + head_2016.getPenyakit());
92     System.out.println("Nomor Antrian   : " + head_2016.getNomorAntrian());
93 }

```

Method CekStatusAntrian

Akan menampilkan “Antrian Kosong!” jika head bernilai null. Jika tidak, tampilkan berapa banyak pasien dalam antrian saat ini, kemudian tampilkan data/status pasien selanjutnya.

```

Pilih menu: 4
|
Saat ini, terdapat 2 pasien.
Status Pasien Selanjutnya:
Nama Pasien      : Suheng
Penyakit Pasien  : Kepala gatal
Nomor Antrian    : 1

```

Tampilan Opsi 4

```

97 static Pasien_2511532016 PanggilPasien_2016(Pasien_2511532016 head_2016) {
98     if(head_2016 == null) {
99         System.out.println("Antrian Kosong!");
100        return null;
101    }
102
103    Pasien_2511532016 pasien_2016 = head_2016;
104    System.out.println(
105        "\nPasien dengan nomor urut " + pasien_2016.getNomorAntrian() +
106        " atas nama " + pasien_2016.getNamaPasien() +
107        " dan memiliki penyakit " + pasien_2016.getPenyakit() +
108        " telah dilayani!"
109    );
110    head_2016 = head_2016.getNext();
111    return head_2016;
112 }

```

Fungsi Panggil Pasien

Menampilkan “Antrian Kosong” jika antrian kosong. Fungsi ini akan memanggil pasien selanjutnya dalam antrian; dalam artian lain, head dari antrian. Kemudian nama, penyakit, serta nomor urut pasien akan ditampilkan.

```

Menu:
1. Daftarkan Pasien
2. Tampilkan Antrian
3. Cari Pasien Dalam Antrian
4. Cek Status Antrian Saat Ini
5. Panggil Pasien
6. Keluar Dari Menu
Pilih menu: 5
Pasien dengan nomor urut 1 atas nama Suheng dan memiliki penyakit Kepala gatal telah dilayani

```

Tampilan Opsi 5

BAB III

KESIMPULAN

Single Linked List (SLL) adalah struktur data yang terdiri dari kumpulan node, di mana setiap node menyimpan data dan pointer next yang menunjuk ke node berikutnya, dengan node terakhir memiliki pointer null sebagai tanda akhir list. Berbeda dengan array yang mengharuskan elemen tersimpan di lokasi memori secara berurutan, SLL memungkinkan node tersebar di lokasi memori mana saja karena keterhubungannya dijaga oleh pointer. Hal ini menjadikan SLL lebih fleksibel dalam hal alokasi memori.

Praktikum ini mengimplementasikan tiga operasi utama pada SLL. Pertama, operasi penambahan node yang dapat dilakukan di depan (`insertAtFront`), di belakang (`insertAtEnd`), maupun pada posisi tertentu (`insertPos`) dengan cara mengatur ulang pointer antar node. Kedua, operasi penelusuran dan pencarian yang dilakukan secara linear dari head hingga node terakhir. Ketiga, operasi penghapusan node yang dapat dilakukan dari depan (`deleteHead`), dari belakang (`removeLastNode`), maupun dari posisi tertentu (`deleteNode`) dengan cara memutus dan menyambung ulang pointer node yang berdekatan.

DAFTAR PUSTAKA

- [1]Oracle. *Java Documentation*. <https://docs.oracle.com/javase/>